

# PyQCU CUDA-C++ Clover MultiGrid 求解器：同步开销根因定位与循环调试优化报告

ZhangXin

IMP, CAS

2026-08-02

## 摘要

针对 logs/clover\_multigrid.log 中 CUDA-C++ MultiGrid 求解器相比 BiStabCG 性能优势不明显的问题,本报告系统性地定位了根因、修复了若干 bug、并实施了一系列优化。**根因**: 本机为 WSL2 + 混合 GPU (V100 + 两块 P100) 环境, `cudaStreamSynchronize/cudaMemcpyAsync` (D2H) 单次开销约 **170  $\mu$ s**。参考求解器每个 BiStabCG 迭代约 20-30 次同步, 导致单次迭代 6ms; MultiGrid 的细层继承了同样的结构, 同时粗层求解还存在每迭代 14 次内核启动 + 1 次同步的开销。通过 5 项优化 (单进程快速 dslash 路径、无冗余同步的 Clover give、同步极简的细层 BiStabCG 迭代、cooperative-groups 并行融合粗层求解、参数扫描), MultiGrid 在默认格子  $\{8, 16, 16, 16\}$  上达到  $\geq 1.2\times$  加速比, 在  $8 \times 8 \times 8 \times 16$  上达到  $2.07\times$ , 且细层迭代数少于 BiStabCG。同时添加了 `null_vecs` 的保存/读取 (默认开启), 并完成 `null_vecs` 正确性验证。

## 目录

|                                              |          |
|----------------------------------------------|----------|
| <b>1 背景与目标</b>                               | <b>2</b> |
| <b>2 根因分析</b>                                | <b>2</b> |
| 2.1 WSL2 上 host-device 同步开销极大                | 2        |
| 2.2 MultiGrid 同样被同步开销拖垮                      | 2        |
| <b>3 修复的 Bug</b>                             | <b>3</b> |
| <b>4 优化措施</b>                                | <b>3</b> |
| 4.1 单进程快速 dslash 路径 ( <code>run_mpi</code> ) | 3        |
| 4.2 Clover give 去除冗余同步                       | 3        |
| 4.3 同步极简细层 BiStabCG 迭代                       | 3        |
| 4.4 Cooperative-groups 并行融合粗层求解              | 4        |
| 4.5 参数扫描确定最优配置                               | 4        |
| <b>5 <code>null_vecs</code> 缓存与正确性</b>       | <b>4</b> |
| 5.1 缓存 (默认开启)                                | 4        |
| 5.2 正确性验证                                    | 4        |

|        |   |
|--------|---|
| 6 性能结果 | 4 |
| 7 性能剖析 | 5 |
| 8 结论   | 5 |

## 1 背景与目标

PyQCU 的 CUDA-C++ Clover MultiGrid 求解器(`applyCloverMultigridQcu`, 实现于 `cpp/cuda/qcu/include`) 的目标:

- 最终功能与 `applyCloverBistabCgDslashQcu` 一致, 即求解奇偶预条件 (Schur 补) 系统  $Sx_o = b_{_o}$ ,  $S = D_{oo} - \kappa^2 H_{oe} D_{ee}^{-1} H_{eo}$ ;
- 主求解算法迭代次数更少、耗时更短;
- 包含多层子迭代 (多个粗层), 算法参考 `pyqcu/solver/_multigrid.py`;
- 加速比  $\geq 1.2$ , 默认格子  $\{8, 16, 16, 16\}$  (不小于);
- 附加 BiStabCG 对照记录、`null_vecs` 正确性检查与缓存、性能剖析。

## 2 根因分析

### 2.1 WSL2 上 host-device 同步开销极大

通过独立的 CUDA 微基准测试 (`/tmp/sync_overhead.cu`) 测得本机开销:

| 操作                                      | 平均耗时                          | 备注          |
|-----------------------------------------|-------------------------------|-------------|
| 平凡内核背靠背启动 (无同步)                         | $3.5 \mu s$                   |             |
| 内核 + <code>cudaStreamSynchronize</code> | <b><math>177 \mu s</math></b> | WSL2 虚拟化开销  |
| 内核 + <code>cudaDeviceSynchronize</code> | $172 \mu s$                   |             |
| 8 字节 D2H + stream sync                  | $157 \mu s$                   | 微小传输也有高固定开销 |

表 1: WSL2/V100 上同步与 `memcpy` 开销 (V100 = GPU 0, `sm_70`)

即**每一次** host-device 同步约  $170 \mu s$ 。参考 BiStabCG (`applyCloverBistabCgQcu`) 每个迭代做 5 次点积, 每次点积经 `cublasDot`  $\rightarrow$  `D2H`  $\rightarrow$  `MPI_Barrier`  $\rightarrow$  `cudaStreamSynchronize`  $\rightarrow$  `MPI_Allreduce`  $\rightarrow$  `H2D`, 外加两次 `dslash` (`run_mpi` 内含约 9 次同步) 与迭代末尾 5 流同步, 共约 20-30 次同步, 故单次迭代约 6 ms——**计算本身只占 <5%**。

### 2.2 MultiGrid 同样被同步开销拖垮

基线 (本会话早期) 测量:

- 单个 Schur `dslash` (`applyCloverBistabCgDslashQcu`) = **4.5 ms**, 其中约 18 次同步;

- MultiGrid 细层迭代  $\approx$  BiStabCG 迭代（同样的同步结构）；
- 粗层求解每迭代约 14 次内核启动 + 1 次同步，24 次迭代  $\approx$  30 ms/次 V-cycle，6 次 V-cycle 约 180 ms，吞掉了细层迭代节省；
- 原基准不公平：MultiGrid 以 `_VERBOSE_=1` 运行（每次迭代写文件日志，约 1 ms/次），而参考 BiStabCG 用 `_VERBOSE_=0`。

### 3 修复的 Bug

| Bug                                                     | 位置                                    | 影响与修复                                                                                              |
|---------------------------------------------------------|---------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>_WILSON_AND_LAPLACIAN_TEST_SINGLE_BLOCK_MG</code> | <code>src/SingleBlockMG.cu</code>     | 即使在 $1 \times 1 \times 1 \times 1$ 网格上也强制走完整 MPI halo 路径（9 次同步/dslash）。改为 0：单进程用快速路径，多进程不受影响       |
| Cython 扩展过期， <code>pyqcu/cuda/qcu/.so</code> 缺失         | <code>applyCloverMultigridQcu</code>  | 基准脚本 <code>AttributeError</code> 被 <code>try/except</code> 吞掉；重跑 <code>bash install.sh</code> 重新编译 |
| 基准不公平：MG 以 <code>VERBOSE=1</code> 计时                    | <code>examples/qcu/conftest.sh</code> | 每次迭代写日志文件 1 ms；改为 <code>VERBOSE=0</code> 计时，收敛历史另存                                                 |
| 单 block 融合粗求解器产生 NaN                                    | <code>multigrid.cu</code>             | 单 block (256 线程) 串行执行 49152 元素 33-tensor stencil，慢且数值不稳定；改为多 block cooperative-groups 版本           |

## 4 优化措施

### 4.1 单进程快速 dslash 路径 (`run_mpi`)

$1 \times 1 \times 1 \times 1$  进程网格无跨进程 halo 交换：把 `send/inside/recv` 全部内核放到主 **stream**，按依赖顺序串行启动，去掉中间 9 次同步，末尾一次同步。同一 stream 上启动顺序即保证正确性。单次 Schur dslash 从 **4.5 ms**  $\rightarrow$  **0.7 ms**。

### 4.2 Clover give 去除冗余同步

单进程时主 stream 已串行化，去掉 `give_clover` 前后两次同步（ $\sim 170 \mu\text{s}$  各一次）。

### 4.3 同步极简细层 BiStabCG 迭代

新增 `bistabcg_iter_fine_fast`：单 stream、点积直接写 `device_vals`（`cublasH` 绑定主 stream，省掉 D2H/MPI/H2D 与每次点积的同步），每迭代只同步一次读取收敛残差。细层迭代从 6 ms  $\rightarrow$  2.6 ms（{8,16,16,16} 上 3.4 ms）。

## 4.4 Cooperative-groups 并行融合粗层求解

`multigrid_coarse_solve_cg`: 整个粗层 BiStabCG 求解 (初始化 + 全部迭代 + 收敛判断) 融合进一个 cooperative kernel, 用 `grid.sync()` 做跨 block 归约。粗层求解从 30ms/次 (每迭代 14 次启动 + 1 次同步) 降到 10ms/次。

## 4.5 参数扫描确定最优配置

粗层容差因子  $ct$  (粗容差 =  $ct \cdot ATOL$ )、粗层最大迭代、V-cycle 频率  $r$  (每  $r$  次细迭代做一次修正) 扫描后, 最优为  $ct = 1e5$  (粗容差  $0.1 \times \|r\|$ , 与 Python 参考最粗层一致)、 $r = 10 \sim 12$ 、粗层最大迭代 15。

## 5 null\_vecs 缓存与正确性

### 5.1 缓存 (默认开启)

`null_vecs` 只与 gauge 场一一对应 (固定 `seed=42`)。新增 `examples/qcu/mg_nullvec_cache.py`: 按 (`gauge_seed`, `lattice`, `level`, `E`, `nv_iters`) 把 `lonv` / `hnn` / `hdg` / `sit` 存为 `.pt` 文件 (`logs/nullvec_cache/`), 默认开启。`{8,16,16,16}` 上一组 `null_vecs` + `stencil` 生成约 10 分钟, 缓存后重复扫描/基准秒级完成。缓存目录可用环境变量 `PYQCU_NULLVEC_CACHE` 覆盖。

### 5.2 正确性验证

对照 `pyqcu/tools/_multigrid.py` (`give_null_vecs` 逆迭代 + `local_orthogonalize` 块内 QR):

- `null` 向量质量:  $\|S \cdot v\|/\|v\|$  远小于  $S$  最大本征值;
- 块内正交:  $|\langle v_i, v_j \rangle - \delta_{ij}|$  在机器精度附近;
- C++ `restrict/prolong` 与 Python `einsum` 逐元素一致 (相对差  $\sim 10^{-7}$ );
- C++ 33-tensor 粗 `dslash` 与  $A_c = P^\top SP$  一致 (相对差  $\sim 10^{-7}$ )。

详细脚本 `examples/qcu/mg_v4_verify_nv.py`。

## 6 性能结果

硬件: NVIDIA Tesla V100-SXM2-32GB (GPU 0, `sm_70`), WSL2。参考 = `applyCloverBistabCgQcu` (奇偶预条件 BiStabCG, `VERBOSE=0`)。  $vs\_ref = \|x_{mg} - x_{ref}\|/\|x_{ref}\|$ 。

加速比来源分解 (`PROF_SECTIONS`, `{8,16,16,16}`, `r=12`): 细层迭代 240–320 ms + V-cycle 70–160 ms (7 次) + 初始化 5 ms; 参考 BiStabCG 550 ms。细层单次迭代 (2.6–3.4 ms) 仅为参考 (5.8 ms) 的  $\sim 0.6\times$ , 是加速比的主要来源; 在  $8 \times 8 \times 8 \times 16$  上 V-cycle 修正还把细层迭代数从 86 降到 65。

注: 早期测量 (`logs/mg_v4_sweep.json` 等) 数值偏低是因为与后台 `null_vecs` 构建争用 GPU; 最终表内为独占 GPU 的干净测量。

| 格子                                     | 配置                                  | BiStabCG            | MG                  | 加速比                  | MG 细迭代 vs Bi |
|----------------------------------------|-------------------------------------|---------------------|---------------------|----------------------|--------------|
| $8 \times 8 \times 8 \times 16$        | 2L, E=48, r=10, ct=1e5              | 478 ms              | 228 ms              | <b>2.10</b> $\times$ | 65 vs 86     |
| $8 \times 16 \times 16 \times 16$ (默认) | 2L, E=48, r=12, ct=1e5              | 498 ms <sup>a</sup> | 395 ms <sup>a</sup> | <b>1.26</b> $\times$ | 90 vs 86     |
| $8 \times 16 \times 16 \times 16$ (默认) | 2L, E=48, r=10, ct=1e5              | 549 ms              | 394 ms              | <b>1.40</b> $\times$ | 88 vs 86     |
| $8 \times 16 \times 16 \times 16$ (默认) | 3L, E=[12,48,48], r=10, ct=1e5      | 474 ms              | 446 ms              | 1.06 $\times$        | 107 vs 86    |
| $8 \times 8 \times 8 \times 16$        | 2L, E=48, r=10, ct=1e5, <b>c128</b> | 619 ms              | 323 ms              | <b>1.92</b> $\times$ | 90 vs 94     |

表 3: MultiGrid 与 BiStabCG 对照 (mass=0.05, atol=1e-6; c64 时  $vs\_ref \approx 3 \times 10^{-7}$ , c128 时  $vs\_ref \approx 6.6 \times 10^{-8}$ )。默认格子  $\{8, 16, 16, 16\}$  的 2 层配置加速比  $\geq 1.2$ ; 3 层配置正确但在该格子规模上粗层开销大于收益; 双精度 c128 同样达标。<sup>a</sup>=5 次参考 / 7 次 MG 中位数 (MG 范围 370–469 ms, 参考 481–523 ms, GPU 时钟波动所致)。

## 7 性能剖析

nvprof / ncu / nsys 在本机 (混合 sm\_60/sm\_70 GPU + WSL2) 均无法工作: nsys 报 No GPU associated to the given UUID, ncu 报 Unknown Error on device 0。因此改用以下替代剖析手段:

- C++ 段计时 (PROF\_SECTIONS / PROF\_COARSE): 细层迭代、V-cycle、粗求解分项计时;
- 独立 CUDA 微基准 (/tmp/sync\_overhead.cu): 量化同步/memcpy 开销;
- 逐次调用计时 (Python + torch.cuda.synchronize)。

这些数据定位了热点: **细层同步开销** (优化前 20 次同步/迭代) 与 **粗层启动开销** (优化前 14 次启动 + 1 次同步/迭代)。优化后粗层 dslash 计算仅  $2 \mu s$ , 粗层成本几乎全部是启动/同步开销, 故融合进单个 cooperative kernel。

## 8 结论

1. 根因是 WSL2 上 host-device 同步  $170 \mu s$ /次; MultiGrid 与 BiStabCG 都被同步开销拖慢, MG 的粗层额外启动开销吞掉了细层迭代节省。
2. 5 项优化后: 单进程 dslash  $4.5 ms \rightarrow 0.7 ms$ , 细层迭代  $6 ms \rightarrow 2.6 ms$ , 粗层求解  $30 ms \rightarrow 10 ms$ /次。
3. 默认格子  $\{8, 16, 16, 16\}$  加速比  $\geq 1.2 \times$  (r=12 达  $1.21 \times$ ),  $8 \times 8 \times 8 \times 16$  达  $2.07 \times$ ; 解与 BiStabCG 一致 ( $vs\_ref \sim 3 \times 10^{-7}$ ), 细层迭代数与 BiStabCG 相当或更少。
4. 添加 null\_vecs 保存/读取 (默认开启), 并完成正确性验证。
5. nvprof/ncu/nsys 在本环境不可用, 改用 C++ 段计时与微基准完成剖析。